TechniView test and validation plan

QA lead: Derek Corniello. **Case owners:** Derek (submissions, scoring, runner and capacity); Ryan Sippy (authoring and UI); Jason Bellerjeau (assignments, access and data); full team (pilot and institutional gates). **Execution window:** CS 5002, spring 2027. **Basis:** Week 9 requirements, technical specification, current testing guide, and campus deployment guide. This plan describes acceptance checks; a `Not Run` case has no claimed result.

1. Scope of testing

Requirement	Component and check this cycle
REQ-01	Web UI, API, worker, go-judge, MySQL: asynchronous Python submission and authorized results.
REQ-02	Worker, MySQL: score finalization, best score, valid attempt count, and infrastructure errors.
REQ-03	Professor UI, authoring API, MySQL, go-judge: draft, canonical solution validation, publication.
REQ-04	Professor/student UI, assignment API, MySQL: published question sets, dates, roster snapshot, visibility.
REQ-05	API and UI: hidden cases, canonical solutions, peer metrics, and role checks.
NFR-01	Application and runner VMs: queue delay, completion time, throughput under a recorded workload.
NFR-02	Dedicated runner VM: CPU, memory, filesystem, capability, and network isolation.
NFR-04	API, MySQL, monitoring: availability over the pilot observation window.
REQ-06, NFR-03, CON-01 to CON-04	Regression, accessibility, cost/privacy and research gates. These are included as supporting checks; the three-cases-per-major rule applies to the eight Must requirements above.

The Could items in the requirements document, additional languages and intelligent hints, are deferred because this release specifies Python execution and no hint service. Automated assignment deadline or grade notifications, collaborative coding, and video proctoring are Won't items and have no implementation contract this cycle. We will not retest Qualtrics itself, campus identity infrastructure, or the browser vendor's TLS implementation; we test TechniView's integration and visible behavior. Known limits are the privileged go-judge container on a separate VM, no target-VM capacity baseline yet, and institutional privacy/research determinations still pending. These are explicit launch gates below.

2. Verification and validation strategy

The team verifies REQ-01 through REQ-05 and NFR-01, NFR-02, and NFR-04 against the Week 9 specification. Unit/API checks cover validation, access control, scoring, transactions, and error bodies. Disposable MySQL and real go-judge integration runs cover worker persistence and sandbox outcomes. System checks use the two-VM staging topology, Caddy HTTPS, browser sessions, and MySQL row locking. D0 and D2 are historical design snapshots where they still mention Judge0 callbacks; the current specification and deployed go-judge path govern these tests.

Students and professors validate the practice and assignment workflows with the faculty advisor. A pilot task script records whether participants can find an assignment, submit, understand permitted feedback, and inspect progress without assistance. Staff review the problem set quality and operations evidence. Qualtrics collects feedback, but participation and consent remain separate from course grades. The advisor reviews the study script, consent process, and results before research enrollment or final acceptance.

Layer	Approximate count at gate	Coverage focus
Unit and API	140	Existing backend/frontend checks plus scoring, validation, errors, and authorization regressions.
Integration	20	Real MySQL, go-judge, worker recovery, transaction and idempotency behavior.
System	8	Browser to HTTPS API, sessions, published assignments, two-VM routing and monitoring.
Acceptance/validation	5	Student and professor tasks, accessibility, advisor and study review.

These are approximate automated/observed checks, not a count of the numbered cases below. The repository currently has more than 130 backend test definitions and a smaller frontend suite; CI runs the routine integration checks, while target-VM capacity and pilot checks require separate runs.

For machine tests, vary concurrent submitters (25, 50, 100), case count (1 and 25), source size (small and 65,535 UTF-8 byte boundary), and the `short`, `cpu-heavy`, `multi-case` workload mix in `backend/performance/workloads/v1.json`. Record p50/p95/p99 queue wait and completion, throughput, errors, queue depth, runner CPU/memory, and host health. Run five measured trials after a warm-up at each accepted load profile. A trial fails if any isolation probe succeeds, a host becomes unhealthy, or the approved performance limits in section 3 are exceeded. The existing `make load-test` command starts a disposable **single-host** integration stack; it is a rehearsal, not target-VM evidence. The target run uses the same request and report fields against the two-VM staging API with a disposable staging database.

For TC-07 on the application VM, set `MYSQL_DB=integration` in the **disposable staging deployment**; the driver refuses any other database name. Set `STAGING_PUBLIC_HOST` to that deployment's DNS name. Then run this command once for warm-up and five times for the measured 100-by-25 trials, saving each JSON report. The public API path goes through Caddy; the metrics scrape stays on the private backend address because Caddy intentionally returns 404 for `/api/metrics`.

```
docker compose --env-file .env.production -f compose.production.yml exec -T \
  -e LOAD_API_BASE_URL="https://$STAGING_PUBLIC_HOST" \
  -e LOAD_STUDENTS=100 -e LOAD_CONCURRENCY=100 \
  -e LOAD_CASES_PER_SUBMISSION=25 \
  -e LOAD_COMPLETION_TIMEOUT_SECONDS=1200 \
  backend python -m performance.load_test
```

For TC-07.2, run the same driver three times concurrently with `LOAD_WORKLOAD=short`, `cpu-heavy`, and `multi-case` and `LOAD_STUDENTS=60`, `20`, and `20` respectively. Start the three commands in one shell and wait for all three exit codes. Keep each report; their combined accepted count must be 100. These shares are the planned test mix, not a measured forecast of real student behavior.

3. Measurable success criteria

Metric	Target threshold	Measurement	Week 9 ID
Queue wait and capacity	At 100 concurrent students and 25 cases/submission, five trials; each trial has p95 queue wait <= 300 s, p95 completion <= 360 s, zero lost case records, and zero infrastructure errors. Team/advisor approve this planning target before it becomes the pilot gate.	Target-VM load report with workload version, VM sizes, worker/runner slots, and timestamps from 202 acceptance through queue start and completion.	NFR-01
Resource isolation	100% of CPU, memory, network, filesystem, and capability probes blocked or terminated at configured limits; zero runner host health failures across five trials.	go-judge execution reports, cgroup v2 counters, denied-access probe output, host telemetry.	NFR-02
API/database availability	>= 99.5% successful one-minute probes over seven consecutive pilot days; both API health and DB dependency must pass. Approve this target before research enrollment.	Timestamped `/api/health` and DB-ready monitoring; numerator is successful minutes, denominator is all scheduled minutes (10,080). Count missing probes as failures.	NFR-04
Scoring accuracy	100% agreement for seeded 0%, 50%, 100%, timeout, and infrastructure outcomes; no duplicate finalization.	Compare submission and `StudentAssignmentProgress` rows against hand-calculated expected values.	REQ-02
Accessibility	Every applicable WCAG 2.1 AA success criterion passes for the core student and professor tasks; zero unresolved failures.	Automated scan plus keyboard, focus, screen-reader, and manual review, with criterion-level evidence.	NFR-03
Task usability	At least 4 of 5 pilot participants in each role complete the assigned core task without facilitator help.	Observe task-script completion and record assistance separately from Qualtrics feedback.	REQ-01, REQ-03, REQ-04

The capacity and availability numbers are proposed acceptance thresholds, not measured claims. If the team/advisor rejects a proposed target, the associated gate is **Blocked** until the approved number is recorded here and in the run sheet; an unapproved or unmeasured target cannot pass.

4. Test cases

Shared fixture for TC-01 to TC-05: Use a disposable migrated MySQL database. Create professor P, enrolled students A and B, and a course owned by P. Create a two-case published Python problem `solve(value)` with public input `[1]` -> `1` and hidden input `[2]` -> 2; `return value` scores 100%, `return 1` scores 50%, and `return 0` scores 0%. Create/publish a question set containing it; create an assignment with `recipient_mode=all`, `available_at` in the past, `due_at` in the future, and `submission_limit=3`, then publish. Record IDs and UTC timestamps in the run sheet. For each submission POST, use a fresh `Idempotency-Key`; poll `GET /api/submissions/{id}` until `done=true` or 60 s, then mark the case Fail if it has not finished. Use a different disposable assignment when a case changes limits or dates. Run real-runner cases against staging; use seeded API fixtures for fast regressions. "App healthy" means `/api/health` returns 200 with `ok=true`; "runner healthy" means its private `/version` returns 200 and the runner container remains running.

ID	Description	Steps	Expected result	Status
TC-01.1	REQ-01: accepted Python	1. Sign in as A. 2. POST fixture problem and `def solve(value): return value` to `/api/submissions` with a fresh key and assignment item ID. 3. Poll returned ID.	POST 202 with ID; terminal result is done and score 1.0; exactly two case rows are stored; hidden input is absent from student response.	Not Run
TC-01.2	REQ-01: syntax failure	1. As A, POST `def solve(value):\n return (` with fresh key. 2. Poll.	POST 202; terminal execution failure, score 0, no server 5xx or leaked hidden input; worker remains healthy.	Not Run
TC-01.3	REQ-01: timeout edge	1. As A, POST `def solve(value):\n while True: pass` with fresh key. 2. Poll. 3. Submit a valid case afterward.	First submission terminates with timeout within configured case wall limit plus 60 s polling allowance; next submission completes; runner VM stays healthy.	Not Run
TC-02.1	REQ-02: first completed attempt	1. Use a fresh assignment. 2. As A, submit 50% source and wait for finalization. 3. Query `StudentAssignmentProgress` for A/item.	`best_score=0.5`, `valid_attempt_count=1`; one progress row.	Not Run

ID	Description	Steps	Expected result	Status
TC-02.2	REQ-02: best score stays maximal	1. On a fresh assignment, submit 50%, 100%, then 0% source, waiting for each finalization. 2. Query progress.	Scores are 0.5, 1.0, 0.0; `best_score=1.0`, `valid_attempt_count=3`.	Not Run
TC-02.3	REQ-02: exhausted limit edge	1. On a fresh item with `submission_limit=1`, finish one graded submission. 2. POST another with a new key. 3. Query progress and submission rows.	Second POST 409 with `submission_limit_reached`; no second submission/progress increment.	Not Run
TC-02.4	REQ-02: ambiguous runner error edge	1. In disposable staging, interrupt the worker after case dispatch using `scripts/verify-direct-runner.sh`. 2. Let recovery finish. 3. Inspect case and progress rows.	Case is recorded as infrastructure/unknown; no silent rerun and no valid-attempt increment for that failed submission.	Not Run
TC-03.1	REQ-03: save professor draft	1. As P, POST `/api/problems` with fixture prompt, starter, canonical source and cases. 2. Read returned ID and query MySQL.	POST 201; one custom problem in `draft` state with the two ordered cases.	Not Run
TC-03.2	REQ-03: publish validated draft	1. As P, create fixture draft. 2. POST `/api/problems/{id}/publish`. 3. Add it to a question set and GET the set.	Publish succeeds; problem state is `published`; set accepts the problem; canonical source is absent from ordinary student reads.	Not Run
TC-03.3	REQ-03: missing canonical edge	1. As P, create draft with `canonical_solution=null` and fixture cases. 2. Attempt publish. 3. Read draft state.	Publish 422; problem stays `draft`; no partial published record.	Not Run
TC-04.1	REQ-04: publish roster assignment	1. As P, POST `/api/courses/{course_id}/assignments` with published question set ID, title, `recipient_mode=all`, start in past and due in future. 2. Publish returned assignment ID. 3. Query recipients.	Create 201 in `draft`; publish 200; A and B each have one recipient row and see the published assignment.	Not Run
TC-04.2	REQ-04: start-time visibility	1. Create an assignment for A with start 60 s in future and publish. 2. As A, GET `/api/courses/{course_id}/assignments`. 3. Wait until 15 s after the recorded start; GET again.	Before start, assignment absent; after start, exactly one visible assignment with the recorded due date.	Not Run
TC-04.3	REQ-04: reversed dates edge	1. As P, POST assignment with `available_at` tomorrow and `due_at` yesterday. 2. Query assignments.	POST 422; no assignment or recipient row created.	Not Run
TC-05.1	REQ-05: staff analytics	1. As P, GET `/api/courses/{course_id}/analytics/overview`. 2. As A, GET same URL.	P receives 200 with aggregate summary/problem items; A receives 403 or concealed 404; overview does not expose individual peer rows.	Not Run
TC-05.2	REQ-05: hidden content edge	1. As A, GET `/api/problems/{problem_id}` and own submission. 2. POST `/api/problems/{problem_id}/validate`. 3. Search response JSON for hidden input and canonical source.	Ordinary reads return 200 without hidden input or canonical source; authoring validation is 403; zero secret fields in A's JSON.	Not Run
TC-05.3	REQ-05: peer submission edge	1. As B, submit fixture problem and record submission ID and B's user ID. 2. As A, GET `/api/submissions/{B_submission_id}` and `/api/courses/{course_id}/analytics/students/{B_user_id}/problems/{problem_id}`.	Peer submission returns 404; peer-only analytics is denied with 403/404; no B metrics in responses.	Not Run
TC-06	REQ-06: retain withdrawn student's history	1. Give B one completed assigned submission. 2. As P, DELETE `/api/courses/{course_id}/members/{B_membership_id}`. 3. Inspect B's membership and progress rows.	DELETE 204; membership has `withdrawn_at`; completed submission and progress rows still exist with their prior values.	Not Run
TC-07.1	NFR-01: peak burst	1. Record VM/image/slot settings on two-VM staging. 2. Run the 100-by-25 command above after warm-up. 3. Save JSON report and DB case count; repeat for five measured trials.	Each trial has 100 accepted submissions and 2,500 persisted case results; each meets section 3 p95 and zero-error limits.	Not Run
TC-07.2	NFR-01: representative mix	1. Run the three concurrent workload commands above on disposable staging. 2. Save all reports and DB case counts. 3. Repeat for five measured trials.	Combined accepted count is 100/trial; reports record p50/p95/p99 completion and p50/p95 queue wait by workload; p95 limits and zero-loss target pass.	Not Run
TC-07.3	NFR-01: overload edge	1. Increase concurrent submitters from 100 to 150, then 200 on disposable staging. 2. Save queue age, failures, host health and recovery times.	Capacity boundary is recorded; no lost/duplicate cases; after load stops, queue drains and both hosts return healthy within 10 min. Overload latency may exceed the 100-user gate.	Not Run
TC-08.1	NFR-02: CPU limit	1. On target runner, run infinite loop with 0.1 s CPU limit via real go-judge integration. 2. Read report and cgroup counters.	Job terminates for CPU limit; no unbounded process remains; runner health remains good.	Not Run
TC-08.2	NFR-02: memory limit	1. Run 256 MiB allocation with 32,768 KiB memory limit. 2. Read report, cgroup events and host health.	Allocation is denied or job is OOM-killed; `bad` sentinel is never printed; host remains healthy.	Not Run
TC-08.3	NFR-02: escape edge	1. Run existing sandbox network, host-path, Docker-socket and capability probes on target runner. 2. Record each probe and host health.	Zero probes reach API/DB/peer or read host secrets/cgroup controls; process has no effective capabilities; host remains healthy.	Not Run
TC-09	NFR-03: accessible core tasks	1. On the pinned browsers and viewports, run student submit/progress and professor author/publish/assign tasks using keyboard and screen reader. 2. Run automated audit; record each applicable WCAG 2.1 AA criterion.	Every applicable criterion passes; zero unresolved failures across the named tasks.	Not Run

ID	Description	Steps	Expected result	Status
TC-10.1	NFR-04: normal monitoring	1. Start two-VM staging. 2. Probe `/api/health` and `/api/health/mysql` once/min for 60 min. 3. Export timestamps.	60 scheduled probes recorded for each route; >= 99.5% successful minutes by section 3 formula.	Not Run
TC-10.2	NFR-04: API outage edge	1. On disposable staging, stop the API for two probe intervals, then restart. 2. Export monitor records.	Both outage minutes count as failures; recovery minutes pass; no missing minute is silently omitted.	Not Run
TC-10.3	NFR-04: DB outage edge	1. On disposable staging, stop MySQL for two probe intervals, then restart. 2. Export monitor records.	DB-dependent health fails for both minutes; recovery passes; failed minutes enter the denominator but not the successful-minute numerator.	Not Run
TC-11	CON-01/CON-02: cost and privacy controls	1. Record campus VM allocation and AWS spend. 2. From outside the app VM, probe MySQL and runner 5050; from app VM, probe runner 5050. 3. Check HTTPS and storage encryption evidence.	UC charge is $0 and AWS spend <= $200/semester; outside probes are denied, app-to-runner works, public HTTP redirects to HTTPS, storage encryption is documented.	Not Run
TC-12	CON-03: research enrollment gate	1. Before inviting participants, inspect the dated UC IRB determination and approved consent script. 2. Compare them to pilot recruitment and data collection steps.	Applicable determination and consent process are on file before first invitation; any mismatch blocks enrollment.	Not Run
TC-13	CON-04: voluntary participation	1. Offer pilot script to an eligible student who declines. 2. Check course grade and required-work records.	Decline creates no research record and changes no course grade or required-work status.	Not Run

The five supporting cases check Should requirements and inherited constraints. They do not substitute for the 25 Must cases above.

5. Test environment

Environment	Hardware, software and configuration	Data and network
Development	Workstation currently runs Arch Linux, Docker 29.9.2, Chromium 156.0.8078.79 and Firefox 159.0. CI uses Node 22; the local frontend Compose image uses Node 20 and the production image uses Node 22. Backend Python 3.14 and SQLite fixtures cover unit/API tests. Local Compose and disposable integration each run on one host; integration uses MySQL 8.0 and go-judge 1.13.3 with Python 3.14.8.	Synthetic fixture above; local loopback ports. No campus firewall claims. Record actual `uname -r`, image digests and browser version with each run.
Staging	Planned OS baseline: Ubuntu 24.04 LTS on two VMs; initial application 4 vCPU/8 GiB; runner 8 vCPU/16 GiB, kernel >= 6.1 with cgroup v2. Pin actual OS release and image SHA in the run sheet before execution; reject a run with an unrecorded version. Caddy serves the built frontend; app VM runs FastAPI, workers, MySQL 8.0, Prometheus/Grafana; runner VM runs go-judge 1.13.3. Browser baseline is Chromium 156.0.8078.79 and Firefox 159.0; desktop 1366x768 and mobile 390x844 viewports.	Disposable seeded profiles, two-case and 25-case problems, draft/published assignments, start/due boundaries, 100 synthetic students. HTTPS 80/443 public; runner 5050 accepts only app VM private IP; MySQL has no host port. Measure baseline RTT and loss, then repeat at 100 ms RTT/1% loss using staging network shaping.
Pilot	Same reviewed image SHA, topology and firewall as staging, with encrypted campus storage and monitoring. Record actual OS, browser, VM sizes and limits before opening enrollment; do not claim staging equivalence until matched.	Real participants only after applicable institutional determination and consent. Synthetic load and deliberate outages stay on staging. One-minute availability probes run for seven consecutive days.

The runner container is privileged to set cgroups, but student jobs use UID/GID 1536, seccomp, no effective capabilities, a read-only root filesystem and isolated network namespace. The VM is the host boundary. No physical sensor or GPU is required for grading tests. Optional Ollama complexity analysis is outside this cycle's Must gates and needs its own GPU capacity assessment if enabled.

6. Pass and fail criteria

For each numbered case, mark **Pass** only after the stated HTTP, database, security, timing and count assertions are observed and evidence is saved. Mark **Fail** for any mismatch, **Blocked** when the environment or required approval is unavailable, and **Not Run** before execution. A 403/404 alternative passes only where the case explicitly permits both. Polling timeouts, missing reports, or unrecorded versions fail or block the case; they do not pass by assumption.

The plan passes when all 25 Must cases pass, section 3 thresholds pass on the target staging VM class, all applicable WCAG criteria pass, there are zero open P1/P2 defects, and the advisor signs the study script. The availability observation must meet 99.5% for all 10,080 scheduled pilot minutes. Any sandbox escape, secret disclosure, incorrect valid-attempt increment after infrastructure failure, missing consent determination, or unresolved privacy control blocks launch. A failed case requires a fix and rerun with fresh evidence; the team records the defect and retest result.

7. Test milestones and activities

Milestone	Test activity (layer)	Owner	Evidence
M7, database and authentication, Week 6	Unit/API during sprint; access and transaction regressions	Jason Bellerjeau, Ryan Sippy	CI output, auth/session and DB test logs.
M3, D1/D2 review, Week 8	System design review before review meeting	Team and course instructor	Reviewed failure paths and updated traceability.
M8, grading pipeline, Week 10	Real MySQL/go-judge integration during sprint; TC-01, TC-02, TC-08	Derek Corniello	Integration reports, recovery trace, sandbox probe output.
M9, authoring and assignments, Week 12	System/UI before demo; TC-03 to TC-05 and accessibility	Ryan Sippy, Jason Bellerjeau	Browser run sheet, API/DB assertions, WCAG findings.
Before M11 live beta, semester 2 Week 8	Target-VM load, isolation and outage rehearsal; TC-07, TC-08, TC-10	Derek Corniello, campus operations	Five load reports, firewall/VM inventory, approved thresholds and launch decision.
M11 live beta, semester 2 Week 8	Acceptance and stakeholder validation	Full team, advisor	Consented task sheets, seven-day uptime log, feedback export.
M12 analytics and research, semester 2 Week 10	Analyze pilot data and recheck metric accuracy before final demo	Full team	Score audit and summary of previously completed capacity results.
M13 CEAS Expo, Week 15	Final acceptance demonstration	Full team and advisor	Signed acceptance record and demonstration checklist.

Run data-integrity and sandbox checks before pilot tasks; unresolved failures stop the later gate.

8. Definition of done

Feature checklist

- [ ] Requirement and component owner are named; API/UI behavior and error codes match the current specification.
- [ ] Relevant unit, integration and edge cases pass; the reviewer can reproduce them from seeded data and recorded versions.
- [ ] Database writes and rollback behavior are checked where applicable; access-controlled responses expose no hidden data.
- [ ] Code, test evidence and user/developer documentation are reviewed and committed; no open P1/P2 defect affects the feature.

Project checklist

- [ ] All Must cases and measurable gates pass; supporting traceability checks are executed and logged.
- [ ] Accessibility audit passes WCAG 2.1 AA for core tasks; target-VM capacity and seven-day availability evidence are retained.
- [ ] No open privacy, access-control or sandbox defect remains; deployment and test run sheets contain actual versions, limits and network settings.
- [ ] The applicable UC research determination and consent process are documented before enrollment; institutional FERPA/retention/deletion review is recorded without claiming an ungranted approval.
- [ ] QA lead, component owners and Faculty Advisor Professor William Hawkins III sign the final review.